

Week 4

Ensemble Learning

for Predictive Modeling

From Individual Models to High-Performance AI Architectures



Bagging & Boosting



Voting & Stacking



Blending & Cascading



Deployment Trade-offs

The Core Philosophy — "Wisdom of the Crowd"

Condorcet Jury Theorem

If every member has a >50% chance of being right, the collective probability of a correct majority decision increases toward 100% as the group grows.

Error Formula

Error = Bias² + Variance + Irreducible Noise

High Bias

Underfitting

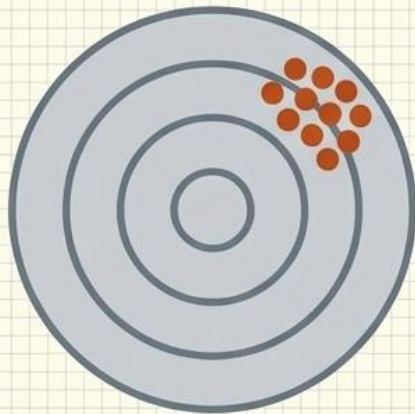
Model too simple — misses real patterns. Fixed by: Boosting

High Variance

Overfitting

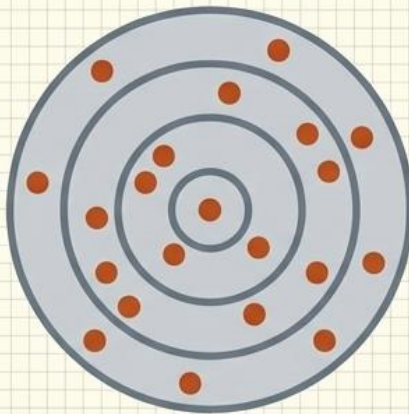
Model too complex — memorizes noise. Fixed by: Bagging

Deconstructing Predictive Error



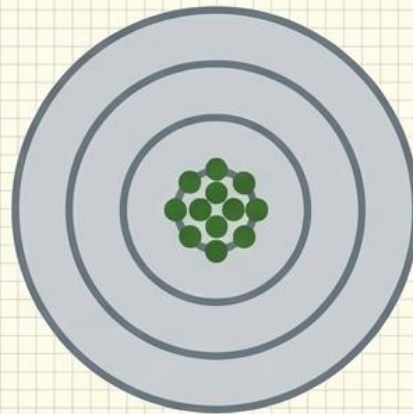
High Bias (Underfitting)

Systematically misses the target in the same direction.



High Variance (Overfitting)

Highly sensitive to noise;
no consistent grouping.



The Goal (Ensembles)

Targeted strikes to reduce
either Bias or Variance.

$$\text{Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Noise}$$

Mission: Every ensemble architecture systematically attacks one of these two variables.

Bagging (Bootstrap Aggregating)



The Concept

Parallel training: Create N subsets of data (with replacement) and train N independent models.



Primary Goal

Reduces Variance — great for high-variance models like deep decision trees.



Key Algorithm

Random Forest = Bagging + Feature Subsampling
(`max_features='sqrt'`)

Python — Scikit-Learn

```
from sklearn.ensemble import RandomForestClassifier

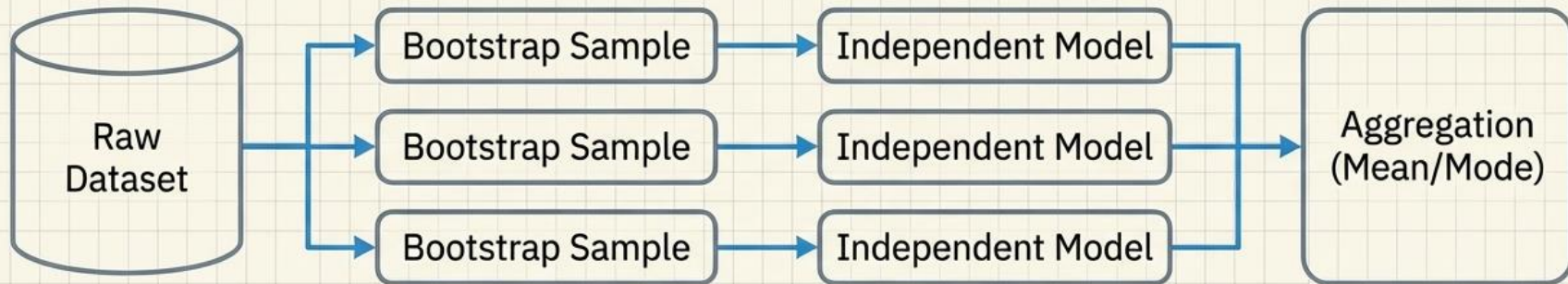
# System view: Ensemble acts as a single object
model = RandomForestClassifier(
    n_estimators=100, max_features='sqrt'
)
model.fit(X_train, y_train)

# Training can be parallelized (n_jobs=-1)
```

Parallel Compute Engines: Bagging

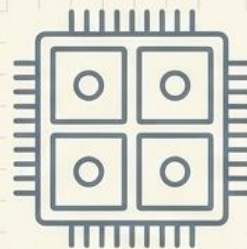
The Architecture

Primary Goal: Reduces Variance



The Software Integration

```
model = RandomForestClassifier(  
    n_estimators=100,  
    n_jobs=-1  
)
```



Hardware Efficiency: Independent models allow for massive parallel training across all available CPU cores.

Boosting (Sequential Learning)

Model A

learns from
errors ↓

Model B

learns from
errors ↓

Model C

Final



Concept

Iterative training: Model B learns from the errors (residuals) of Model A.



Primary Goal

Reduces Bias — when you need the highest possible accuracy on tabular data.



Key Systems

XGBoost, LightGBM, CatBoost — the heavy hitters of tabular AI.



Trade-off

High accuracy but higher latency. Sequential nature prevents full parallelization.

Implementing Boosting in Pipelines

Early Stopping

Automatically halts training when validation loss stops improving
— prevents overfitting in production.

Config-Driven Parameters

In production systems, params dict is loaded from external .yaml or .json files for reproducibility.

DMatrix Format

XGBoost's native optimized data format — faster training and lower memory usage than pandas DataFrames.

Python — XGBoost

```
import xgboost as xgb
# System parameters (usually loaded from yaml)
params = {
    'objective': 'binary:logistic',
    'eval_metric': 'logloss',
    'max_depth': 6
}

dtrain = xgb.DMatrix(X_train, label=y_train)
# Watchdog = validation set for early stopping
bst = xgb.train(params,
                dtrain, num_boost_round=100)

# Train with early stopping
model.fit(
    X_train,
    y_train,
    eval_set=[(X_val, y_val)],
    early_stopping_rounds=20, # stop if no improvement for 20 rounds
    verbose=True
)
```

Voting Ensembles (Hard vs. Soft)

Hard Voting

Majority rule: If 2/3 models predict Class A, output is A.

✓ Simple, interpretable

✗ Ignores model confidence

Formula: $P_{\text{final}}(y) = (1/n) \times \sum w_i \times P_i(y)$
where w_i is an optional weight per model



Tip:

Voting Classifiers are ideal for combining diverse model types (e.g. NLP + Tabular models). Use weights=[1,1,2] to give stronger models more influence.

Soft Voting

Average predicted probabilities — accounts for confidence.

✓ More robust, generally better

✗ Requires probability outputs

```
from sklearn.ensemble import VotingClassifier

# Sub-modules defined previously (lr, dt, svc)
eclf = VotingClassifier(
    estimators=[('lr', clf1), ('dt', clf2), ('svc', clf3)],
    voting='soft',
    weights=[1, 1, 2]
)
eclf.fit(X, y)
```


Stacking & Meta-Learners

RF

SVM

XGB

KNN

Level-0 (Base Learners)

OOF Predictions ↓

Meta-Learner (Level-1)

Final Prediction ↓

Output ✓

⚠ The Leakage Problem

You cannot train the Meta-Learner on the same data used by Base Learners → use Out-of-Fold (OOF) predictions instead.

```
from sklearn.ensemble import StackingClassifier

estimators = [
    ('rf', RandomForestClassifier()),
    ('ext', ExtraTreesClassifier()),
]

stack_model = StackingClassifier(
    estimators=estimators,
    final_estimator=LogisticRegression()
)
```

Out-of-Fold (OOF) prediction refers to predictions generated for training samples using models that were **not trained on those specific samples**, typically within a **K-fold cross-validation framework**.

It is widely used in **stacking ensembles**

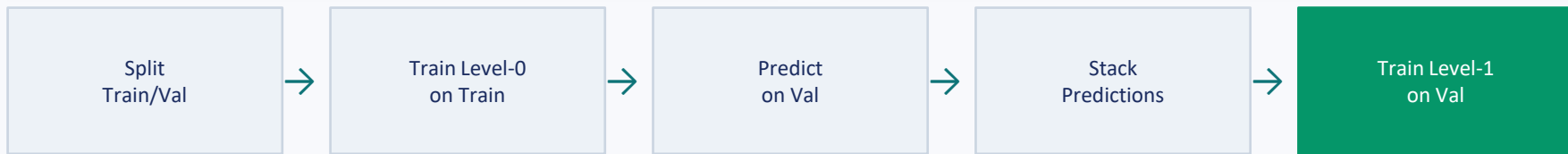
Blending — The Faster Cousin of Stacking

Stacking

Uses cross-validation (OOV). More data-efficient, less prone to overfitting. Preferred for competition.

Blending

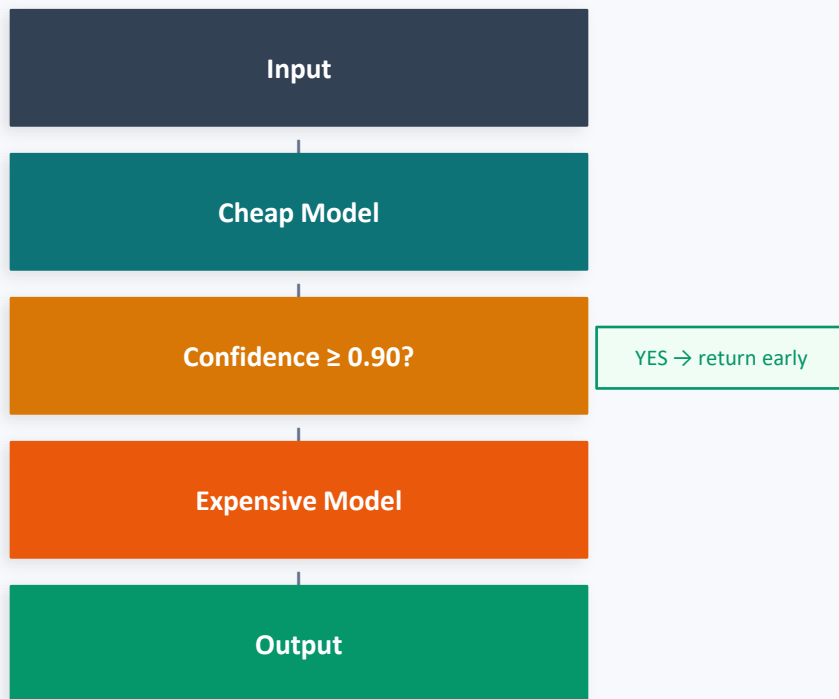
Uses a single hold-out validation set. Faster to implement, but wastes some data.



Python — Manual Blending Logic

```
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2)
model_a = RandomForestClassifier().fit(X_train, y_train)
pred_a = model_a.predict_proba(X_val); pred_b = model_b.predict_proba(X_val)
X_blend = np.column_stack((pred_a, pred_b))
meta_learner = LogisticRegression().fit(X_blend, y_val)
```

Cascading & Auto-Ensembling



Python — Cascading Inference

```
def cascading_inference(input_data,
    cheap_model, expensive_model,
    threshold=0.90):
    probs = cheap_model.predict_proba(input_data)
    if np.max(probs) >= threshold:
        return probs, 'Cheap-Path'
    return expensive_model.predict_proba(
        input_data), 'Expensive-Path'
```



AutoGluon

Multi-layer Stack Ensembling
with Distillation.



H2O.ai

AutoML — builds massive
Stacked Ensembles by default.

System Design — The Ensemble Module

An ensemble is a structural design pattern — the Composite Pattern — not just a math trick.



Uniform Interface

Ensemble implements the same `predict()` as a `SingleModel`. The API/UI doesn't know if it's 1 model or 50.



Concurrency

Use Python multiprocessing or async calls to run Level-0 predictions in parallel — not sequentially.



Graceful Degradation

If 1 of 5 models times out, the system ignores it and proceeds with remaining model votes.



Payload Management

Pass pointers or use shared memory to avoid bottlenecks when routing data to multiple models.

Deployment & Lifecycle Trade-offs

Metric	Single Model	Ensemble
Accuracy	Baseline	Higher ↑
Inference Speed	Fast ✓	Slower ✗
Maintenance	Easy ✓	Hard (many models) ✗
Explainability	High ✓	Low (Black Box) ✗

Rule of Thumb:

Only use ensembles when the cost of a wrong prediction outweighs the cost of extra compute and maintenance overhead.



Inference Latency

10 models take 10× longer. Can your API SLA handle it?



Model Drift

Monitor drift for every sub-model. One failing model degrades the whole ensemble.



Version Control

10 weight files to version, test, and deploy instead of one.

Comparison of Ensemble Architectures

Technique	Primary Goal	Training Style	Complexity	Best Use Case
Bagging	Reduce Variance	Parallel (Independent)	Low	High-variance models (Deep Trees)
Boosting	Reduce Bias	Sequential (Dependent)	Medium	Tabular data — accuracy matters
Voting	Neutralize Error	Parallel	Low	Combining diverse model types
Stacking	Maximize Accuracy	Layered (OOF)	High	Competitions / offline batch
Blending	Speed & Simplicity	Layered (Hold-out)	Medium	Large datasets, slow CV
Cascading	Efficiency	Conditional	High	High-traffic APIs (FaceID, Spam)

Summary & Project Milestones

Recap — When to use which ensemble technique



Bagging

Use for: Stability

Parallel, independent models. Great for reducing variance in deep trees.



Boosting

Use for: Performance

Sequential learning. Best accuracy on tabular data. Use XGBoost/LightGBM.



Stacking

Use for: Complexity

Hierarchical meta-learner. Use OOF to avoid leakage. Best for competitions.



Cascading

Use for: Efficiency

Short-circuit pattern. High-throughput APIs. Cheap model as gatekeeper.

Project Tip: Start with a single strong baseline model. Only move to Ensembles once your pipeline is stable.